

Data Source - [NYC Trip Data](#)

We will analyse **2025** year data -> **Green Taxi** trip records only.

Project Steps

Step 1 – Download data from the above website

Green Taxi Trip Records (2026) in Parquet format – We will retrieve the data directly using ADF (**HTTP connection**)

Download **TaxiZone Lookup table** (csv format).

Step 2 – Set up MS Azure (BRONZE LAYER)

- Create a **resource group, storage account** (data lake), 3 **containers** inside the storage account – *gold, silver, bronze*
- In the bronze container, create a directory (**Add Directory** option) – *trip_zone*. Upload the **TaxiZone Lookup table** csv file in this folder.
- Create **ADF resource**
- Create **Databricks resource**
- Create **Microsoft Entra Id (service principal)** to give access to Databricks to Data lake

Step 3 – Create a pipeline in ADF (to ingest data from the **NYC link** to the Bronze container)

The site has data in **Parquet** format separated by each month.. i.e each month has different links –

Jan - https://d37ci6vzurychx.cloudfront.net/trip-data/green_tripdata_2025-01.parquet

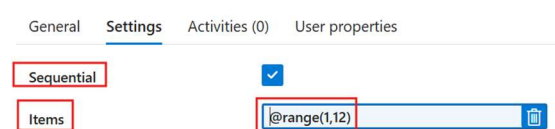
Dec - https://d37ci6vzurychx.cloudfront.net/trip-data/green_tripdata_2025-12.parquet

Base url – highlighted in blue

Relative url – highlighted in yellow

We will use **For Each loop** activity which will return 1, 2, 3, 4... as the output.

i. Configure ForEach loop activity



ii. Configure Copy activity

-> Add Copy activity inside ForEach (**Activities**)

Source dataset – HTTP, Parquet format

Source ls – Update the base url

Configuring parametrized relative url

-> Add dynamic content -> Create a parameter *param_month_no* -> Update **exp** as below –

trip-data/green_tripdata_2025-0@{dataset().param_month_no}.parquet

Connection Schema Parameters

Linked service * ls_http Test connection Edit
Connection successful

Base URL https://d37ci6vzurychx.cloudfront.net/

Relative URL trip-data/green_tripdata_2025-0@{data} Preview data

Compression type snappy

Pipeline expression builder

Add dynamic content below using any combination of [expressions](#), [functions](#) and [system variables](#).

trip-data/green_tripdata_2025-0@{dataset().param_month_no}.parquet

-> In the above exp, we are dynamically replacing the month no with the parameter that we have created.

Original rel url was - **trip-data/green_tripdata_2025-01.parquet**. So, we are replacing number 1 with the parameter created.

-> **Parameter value** will be the #time ForEach loop is running i.e update the exp as **@item()**.

Sink dataset – Azure Data Lake, Parquet format

Sink ls – Update the base url

General Source Sink Mapping Settings User properties

Source dataset * ds_http Open New Preview data

Dataset properties

Name	Value
param_month_no	@item()

-> When you run the above pipeline, it succeeds for months 1-9. However, it fails for 10-12.

-> To handle this, we need to add If Condition activity which will check the month no. and run the related copy activity accordingly.

ii. Configure If-Condition activity

-> Add **If-Condition activity** inside ForEach loop. Update **exp** to check if ForEach iteration is greater than 9 – **@greater(item(),9)**

-> Copy the Copy activity that we configured earlier under **False** condition

-> Again, copy the same **Copy activity** under **True** condition. However, we need to create a **new source dataset without 0 to modify the relative url field**.

Base URL https://d37ci6vzurychx.cloudfront.net/

Relative URL trip-data/green_tripdata_2025-0@{data}

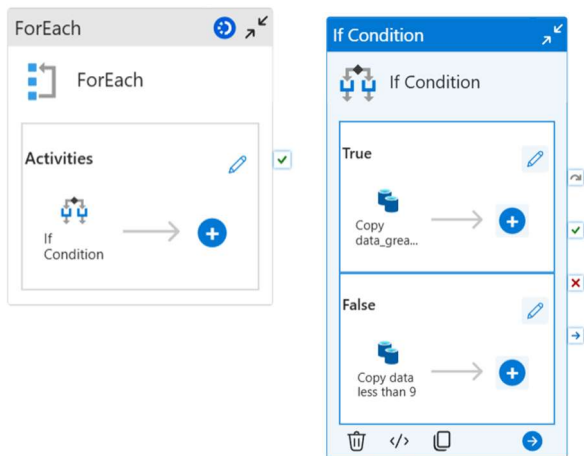
Create a parameter - **param_month_greaterthan9**. Then update the exp for relative url as below -

trip-data/green_tripdata_2025-@{dataset().param_month_greaterthan9}.parquet

Parameter value -> Update **@item()**

Pipeline sequence

For Each activity -> If Condition activity (inside ForEach) -> Copy activity (True) -> Copy activity (False)



Parameters	Variables	Settings	Output							
Activity name	↑↓		Activity status	↑↓	Activity ...	↑↓	Run start	↑↓	Duration	↑↓
Copy data_greater_than_9			✔ Succeeded		Copy data		6/15/2026, 10:52:47 AM		16s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:52:46 AM		17s	
Copy data_greater_than_9			✔ Succeeded		Copy data		6/15/2026, 10:52:29 AM		16s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:52:29 AM		18s	
Copy data_greater_than_9			✔ Succeeded		Copy data		6/15/2026, 10:52:12 AM		15s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:52:12 AM		17s	
Copy data less than 9			✔ Succeeded		Copy data		6/15/2026, 10:51:56 AM		15s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:51:55 AM		17s	
Copy data less than 9			✔ Succeeded		Copy data		6/15/2026, 10:51:39 AM		15s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:51:38 AM		17s	
Copy data less than 9			✔ Succeeded		Copy data		6/15/2026, 10:51:21 AM		15s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:51:20 AM		18s	
Copy data less than 9			✔ Succeeded		Copy data		6/15/2026, 10:51:02 AM		17s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:51:01 AM		19s	
Copy data less than 9			✔ Succeeded		Copy data		6/15/2026, 10:50:44 AM		16s	
If Condition			✔ Succeeded		If Condition		6/15/2026, 10:50:43 AM		18s	

Step 4 – Create Azure Databricks resource in MS Azure (SILVER LAYER)

Step 5 – Create compute in Databricks

Step 6 – Create Microsoft Entra Id (this will allow databricks to access the data in the data lake)

-> We will create a **service application** which will access data in the azure data lake.

Step 7 – Launch Databricks Workspace

Go to Workspace and create a folder – *NYC_Project*

Create a new Notebook -> Name it as *Silver Layer Transformations*.

i. Run the below code to access data in the data lake -

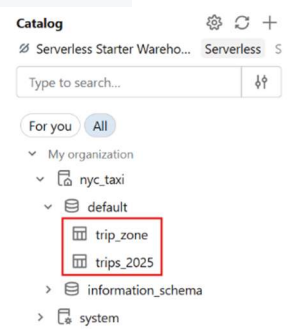
```
spark.conf.set('fs.azure.account.auth.type.<storage-account>.dfs.core.windows.net',
'OAuth')
spark.conf.set('fs.azure.account.oauth.provider.type.<storage-
account>.dfs.core.windows.net',
'org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider')
spark.conf.set('fs.azure.account.oauth2.client.id.<storage-account>.dfs.core.windows.net',
'<application-id>')
spark.conf.set('fs.azure.account.oauth2.client.secret.<storage-
account>.dfs.core.windows.net', 'service_credential')
spark.conf.set('fs.azure.account.oauth2.client.endpoint.<storage-
account>.dfs.core.windows.net', 'https://login.microsoftonline.com/<directory-
id>/oauth2/token')
```

iii. Perform transformations in the dataset.

iv. Write the data to MS Azure – **Silver** folder.

v. Create **managed Delta tables** in Databricks

```
df_trips.write.format("delta") \
.mode("append") \
.saveAsTable("nyc_taxi.default.trips_2025")
```



Step 8 - Connect to Power BI

-> Connect to Power BI using **Partner Connect**.

Advanced Editor

trips_2025

Display Options ▾

```
let
    Source = Databricks.Catalogs("adb-7405612338950389.9.azuredatabricks.net", "/sql/1.0/warehouses/66ac7eb99052e435", [Catalog = "",
        Database = ""]),
    nyc_taxi_Database = Source{[Name="nyc_taxi",Kind="Database"]}[Data],
    default_Schema = nyc_taxi_Database{[Name="default",Kind="Schema"]}[Data],
    trips_2025_Table = default_Schema{[Name="trips_2025",Kind="Table"]}[Data]
in
    trips_2025_Table
```

> trip_zone

✓ trips_2025

- ☐ dropoff_date
- ☐ ∑ Dropoff_Location_ID
- ☐ dropoff_time
- ☐ ∑ payment_type
- ☐ pickup_date
- ☐ ∑ Pickup_Location_ID
- ☐ pickup_month
- ☐ pickup_time
- ☐ ∑ pickup_year
- ☐ ∑ total_amount
- ☐ ∑ trip_distance
- ☐ ∑ trip_type
- ☐ ∑ VendorID